Linguaggi di Programmazione

Alberi e ambiguità

Roberto Gorrieri

Alma Mater Studiorum - Università di Bologna

Alberi di derivazione e alberi sintattici

Derivazioni e alberi

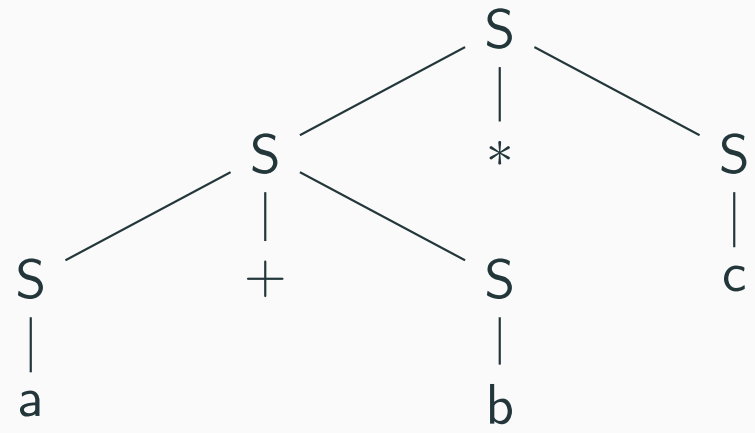
Consideriamo la grammatica

$$S \to a \mid b \mid c \mid S + S \mid S * S$$

Consideriamo la derivazione

$$S \Rightarrow \underline{S} * S \Rightarrow \underline{S} + S * S \Rightarrow a + \underline{S} * S \Rightarrow a + b * \underline{S} \Rightarrow a + b * c$$

- **derivazione leftmost** (o canonica "sinistra"): ad ogni passo riscriviamo il nonterminale più a sinistra
- è possibile associarle un **albero di derivazione**



Derivazioni e alberi

Consideriamo un'altra derivazione per $a + b * c$:

$$S \Rightarrow S * \underline{S} \Rightarrow \underline{S} * c \Rightarrow S + \underline{S} * c \Rightarrow \underline{S} + b * c \Rightarrow a + b * c$$

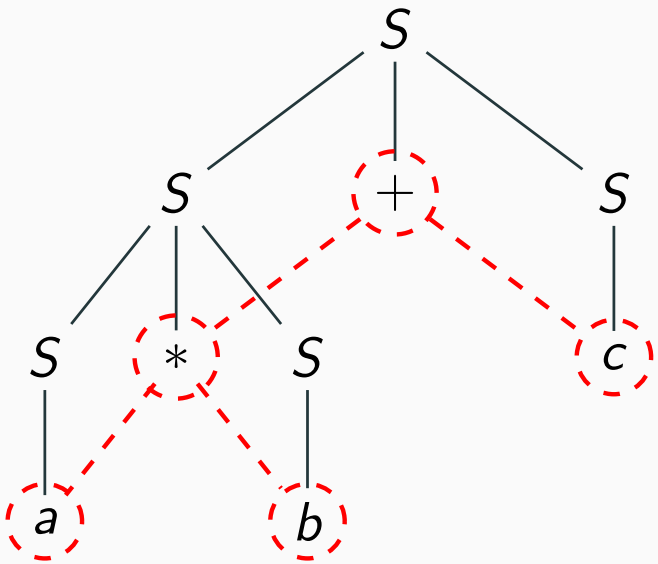
- **derivazione rightmost** (o canonica "destra"): ad ogni passo riscriviamo il nonterminale più a destra
- Genera lo stesso albero di derivazione della pagina precedente

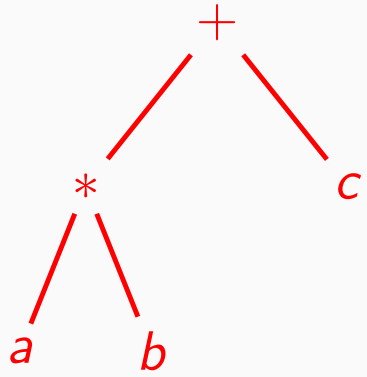
Un albero di derivazione riassume tante derivazioni diverse, ma tutte equivalenti (nel senso che tutte generano lo stesso albero)
Esercizio: Trova almeno un'altra derivazione per $a + b * c$ che genera lo stesso albero di derivazione.

Albero di derivazione

- Esiste una corrispondenza biunivoca tra derivazioni sinistre/leftmost (o destre/rightmost) e alberi di derivazione, nel senso che:
  - Dato un albero di derivazione, esiste una sola derivazione **leftmost** che lo genera
  - Dato un albero di derivazione, esiste una sola derivazione **rightmost** che lo genera
  - Data una derivazione leftmost (o rightmost), ad essa viene associato **univocamente** un albero di derivazione
- L'albero di derivazione fornisce informazioni semantiche: quali operandi per quali operatori

Alberi Sintattici da Alberi di Derivazione

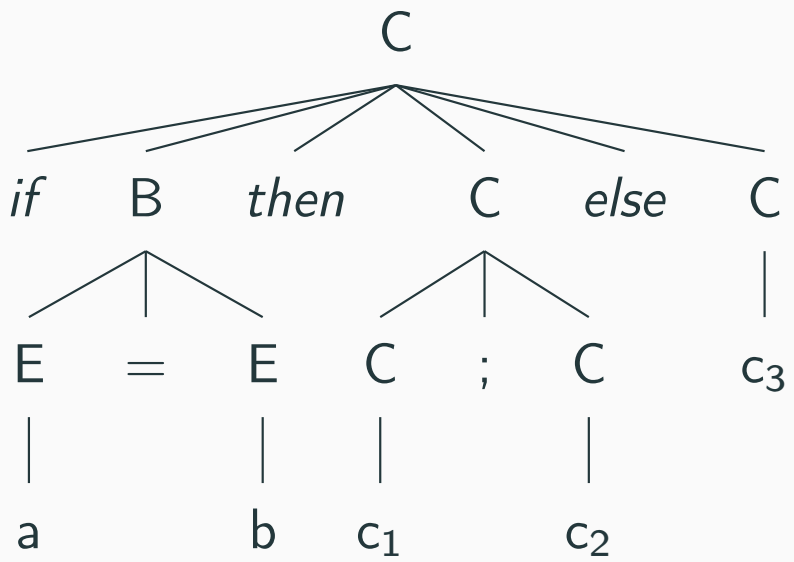




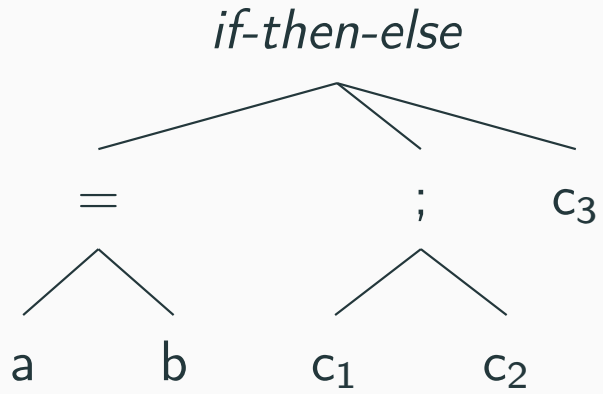
- L'albero sintattico è un albero di soli terminali dove i nodi interni sono etichettati da operatori e le foglie sono gli operandi.
- In questo caso, mi fornisce l'informazione semantica che bisogna prima calcolare $a * b$ e poi sommargli c, cioè calcolare $(a * b) + c$.
- Tipica rappresentazione interna usata dal compilatore per generare codice intermedio.

Esempio

$C \to \text{if } B \text{ then } C \text{ else } C \mid C; C \mid \text{skip} \mid \ldots$ (comandi)
$B \to E = E \mid B \textbf{ and } B \mid \ldots$ (espressioni booleane)
$E \to a \mid b \mid E + E \mid \ldots$ (espressioni aritmetiche)



Albero di derivazione



Albero sintattico

(c_1, c_2 e c_3 sono comandi elementari non specificati)

Albero di derivazione

Definizione di albero di derivazione
Data una grammatica libera $G = (NT, T, R, S)$, un albero di derivazione (o di parsing) è un albero ordinato in cui:

- ogni nodo è etichettato con un simbolo in $NT \cup T \cup \{\epsilon\}$;
- la radice è etichettata con S;
- ogni nodo interno è etichettato con un simbolo in NT;
- se il nodo n
  - ha etichetta $A \in NT$ e
  - i suoi figli sono nell'ordine $m_1, ..., m_k$ con etichetta $X_1, ..., X_k$ (in $NT \cup T$), allora $A \to X_1 ... X_k$ è una produzione in R;
- se il nodo n ha etichetta ϵ, allora n è una foglia, è figlio unico, e se suo padre è etichettato A, allora $A \to \epsilon$ è una produzione di R;
- se inoltre ogni nodo foglia è etichettato su $T \cup \{\epsilon\}$, allora l'albero corrisponde ad una derivazione completa (cioè una derivazione che ha generato una parola).

Teorema

Dato che $w \in T^*$ appartiene a $L(G)$ se $S \Rightarrow^* w$ (ovvero se posso derivarla a partire dal simbolo iniziale S), ne segue che $w \in T^*$ appartiene a $L(G)$ se esiste un albero di derivazione completo per w.

Teorema
Una stringa $w \in T^*$ appartiene a $L(G)$ se e solo se ammette un albero di derivazione completo le cui foglie lette da sinistra a destra diano la stringa w.

(cioè l'albero viene visitato in ordine anticipato tralasciando i nonterminali)

Ambiguità

Il problema dell'ambiguità

Consideriamo la grammatica

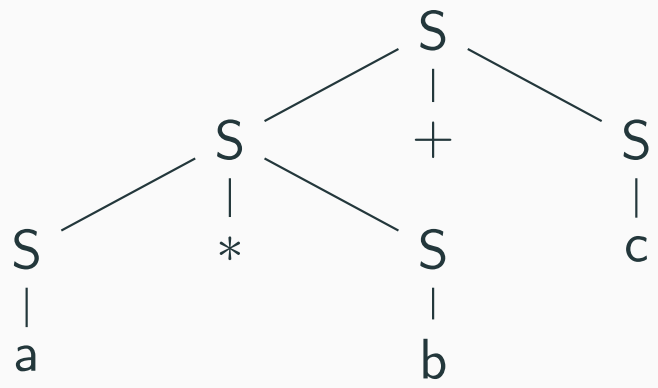
$$S \to a \mid b \mid c \mid S + S \mid S * S$$

Esistono **due derivazioni leftmost** per $a * b + c$:
$S \Rightarrow \underline{S} + S \Rightarrow \underline{S} * S + S \Rightarrow a * \underline{S} + S \Rightarrow a * b + \underline{S} \Rightarrow a * b + c$
$S \Rightarrow \underline{S} * S \Rightarrow a * \underline{S} \Rightarrow a * \underline{S} + S \Rightarrow a * b + \underline{S} \Rightarrow a * b + c$
E quindi **due alberi di derivazione diversi!** (vedi prossima slide)

- Per questa grammatica, la stringa $a * b + c$ ha più di un albero di derivazione, quindi la grammatica è **ambigua** (inutilizzabile per dare semantica ad $a * b + c$).
- Bisogna utilizzare grammatiche non ambigue, o manipolare grammatiche ambigue per disambiguarle (problema non facile e non sempre risolvibile).

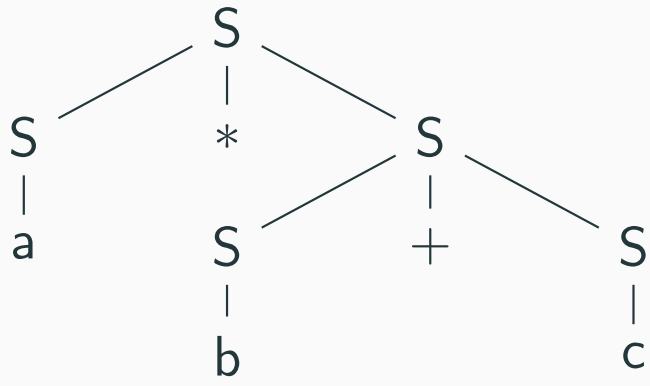
Le due derivazioni leftmost e i due alberi di derivazione

$$S \Rightarrow \underline{S} + S \Rightarrow \underline{S} * S + S \Rightarrow a * \underline{S} + S \Rightarrow a * b + \underline{S} \Rightarrow a * b + c$$



Corrisponde a calcolare $(a * b) + c$

$$S \Rightarrow \underline{S} * S \Rightarrow a * \underline{S} \Rightarrow a * \underline{S} + S \Rightarrow a * b + \underline{S} \Rightarrow a * b + c$$



Corrisponde a calcolare $a * (b + c)$

Grammatica ambigua e linguaggio ambiguo

1. Una grammatica libera G è **ambigua** se $\exists w \in L(G)$ che ammette più alberi di derivazione.
2. Un linguaggio L è **ambiguo** se tutte le grammatiche G tali che $L(G) = L$ sono ambigue.

Alcune grammatiche possono essere manipolate in modo da rimuovere l'ambiguità e generare lo stesso linguaggio. Ad esempio

$$G = \begin{cases} S \to A \mid \epsilon \\ A \to \epsilon \end{cases}$$

G è ambigua perché $S \Rightarrow \epsilon$ ed anche $S \Rightarrow A \Rightarrow \epsilon$ (due derivazioni leftmost per la stessa stringa!)
Tuttavia esiste G' non ambigua tale che $L(G') = L(G) = \{\epsilon\}$:

$$G' = S \to \epsilon$$

Esistono grammatiche patologiche dalle quali non è possibile rimuovere l'ambiguità. Il linguaggio generato da queste grammatiche è ambiguo!

Esempio di linguaggio ambiguo (1)

Sappiamo che $L_1 = \{a^n b^n | n \geq 1\}$ è libero perchè generato dalla grammatica non ambigua $G_1 = S_1 \to aS_1 b \mid ab$
Non è difficile immaginare che

$$L_2 = \{a^n b^n c^m d^m | n, m \geq 1\}$$

sia libero non ambiguo ($G_2 = S_2 \to S_1 S_1'$ e $S_1' \to cS_1' d \mid cd$)
Ed anche

$$L_3 = \{a^n b^m c^m d^n | n, m \geq 1\}$$

sia libero non ambiguo ($G_3 = S_3 \to aS_3 d \mid aS_3' d$ e $S_3' \to bS_3' c \mid bc$)

Esempio di linguaggio ambiguo (2)

È facile verificare che $L_4 = L_2 \cup L_3$ è un linguaggio libero (perché l'unione di due linguaggi liberi è un linguaggio libero: infatti $G_4 = S_4 \to S_2 \mid S_3$ genera L_4).

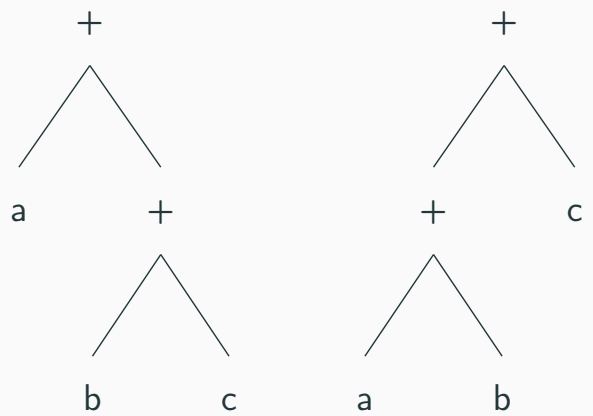
- Tuttavia si può dimostrare che L_4 è ambiguo!
- Infatti, tutte le stringhe del tipo $a^n b^n c^n d^n$ hanno una doppia derivazione leftmost (e quindi hanno due differenti alberi di derivazione):
  - una attraverso le produzioni che generano il linguaggio L_2,
  - ed un'altra attraverso le produzioni che generano il linguaggio L_3.

N.B.: Questa argomentazione non dipende dalla scelta delle grammatiche usate per generare L_2 e L_3.

Rimuovere l'ambiguità

Non è sempre possibile rimuovere l'ambiguità. Tuttavia in alcuni casi si può. Vediamo come rimuovere l'ambiguità dalla grammatica delle espressioni aritmetiche. Infatti: $S \to a \mid b \mid c \mid S + S \mid S * S$ è ambigua (vedi pagina 12 = slide 9). Quali problemi ci sono?

1. **Precedenza** del $*$ rispetto al $+$ in modo che $a * b + c$ sia interpretato correttamente come $(a * b) + c$ (e non come $a * (b + c)$).
2. **Associatività** del $+$ e del $*$ (anche se sembra non far differenza semantica, è comunque fonte d'ambiguità). Quale albero si deve associare all'espressione $a + b + c$?



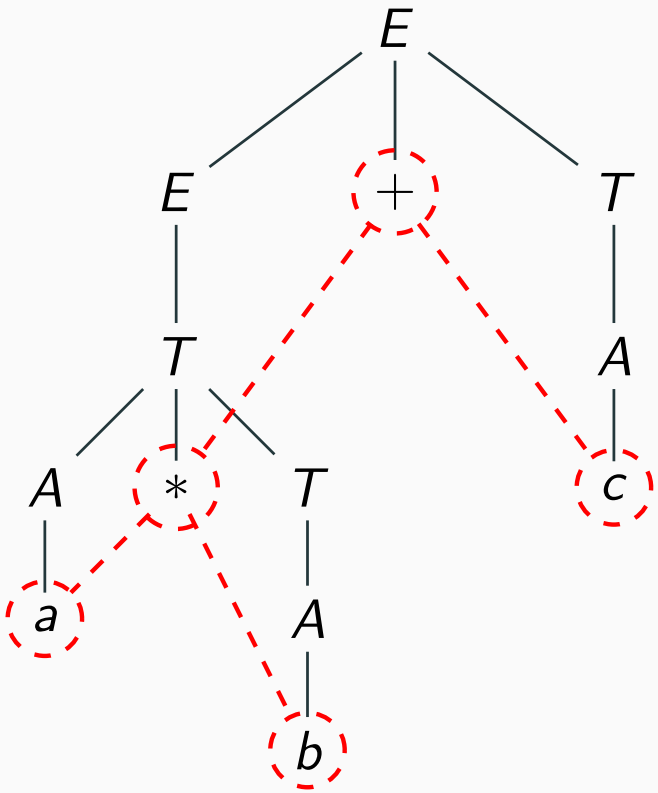
Rimuovere l'ambiguità (2)

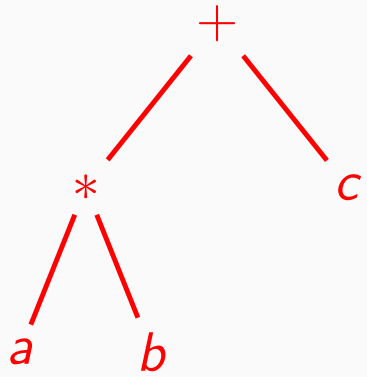
Per disambiguare la grammatica con simbolo iniziale S, modifichiamo la grammatica così:

$$\begin{cases} E \to E + T \mid T \\ T \to A * T \mid A \\ A \to a \mid b \mid c \end{cases}$$

- Abbiamo fissato le **associatività**:
  - La produzione $E \to E + T$ dice che il $+$ associa a sinistra.
  - La produzione $T \to A * T$ dice che il $*$ associa a destra.
- Abbiamo fissato la **precedenza** fra i due operatori:
  - Dato che il simbolo iniziale è E e l'operatore $*$ viene generato da T, gli alberi sintattici che vengono generati avranno il $*$ al di sotto del $+$ (come per $(a * b) + c$).
  - Per essere più precisi: non potranno mai avere il $+$ al di sotto del $*$ (cioè non potranno generare alberi del tipo $a * (b + c)$).

Unico albero derivabile per $a * b + c$





Unico albero sintattico derivabile per $a * b + c$ dalla grammatica con simbolo iniziale E.

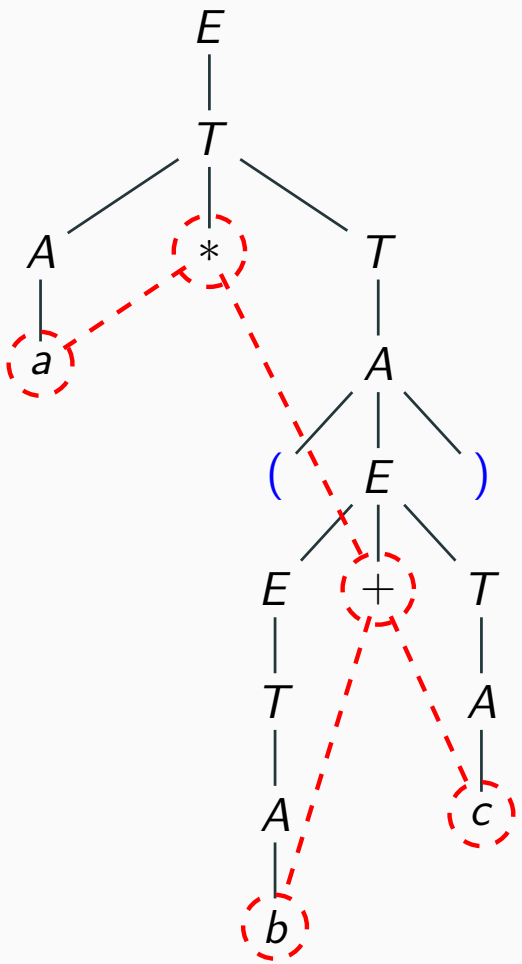
Rimuovere l'ambiguità (3)

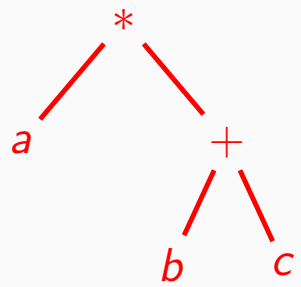
- Nota che $L(E) = L(S)$, quindi la nuova grammatica è equivalente a quella originale e in più è non ambigua.
- Ma non è semanticamente equivalente a quella originale!
  - L'albero sintattico in cui il $*$ sta sopra al $+$ (cioè quello per $a * (b + c)$) non è generabile!
- Allora, per ottenere (quasi) lo stesso linguaggio, ma sicuramente gli stessi alberi sintattici, modifichiamo la grammatica così

$$\begin{cases} E \to E + T \mid T \\ T \to A * T \mid A \\ A \to a \mid b \mid c \mid (E) \end{cases}$$

Le parentesi (,) sono nuovi terminali ausiliari usati solo per disambiguare (zucchero sintattico), ma non sono semanticamente rilevanti.

Unico albero derivabile per $a * (b + c)$





Le parentesi sono nuovi terminali della grammatica aumentata, ma non servono per costruire l'albero sintattico astratto (o albero di sintassi astratta), che è un albero sintattico della grammatica originale!

Sintassi astratta e sintassi concreta

- **Sintassi Astratta**: grammatica semplice ed intuitiva, ma ambigua (come per la grammatica con simbolo iniziale S).
- **Sintassi Concreta**: grammatica (spesso complessa, ma) non ambigua, che fa uso di zucchero sintattico (parentesi, begin-end, e similari), adatta al parsing, perchè associa univocamente un albero di derivazione per ogni stringa (come per la grammatica con simbolo iniziale E).

Dall'albero di derivazione da sintassi concreta estraiamo un albero sintattico senza zucchero sintattico, che corrisponde ad un **albero sintattico della sintassi astratta**, detto albero sintattico astratto, o albero di sintassi astratta (abstract syntax tree)
N.B. Un albero sintattico non è mai ambiguo!

Esercizio: espressioni booleane

- Si consideri la seguente sintassi astratta per le espressioni booleane:
  - $B \to a \mid b \mid \neg B \mid B \vee B$
- La grammatica è ambigua? Suggerimento: considerate la stringa $\neg a \vee b$
- Trovare una grammatica equivalente non ambigua, supponendo che
  - l'operatore $\neg$ abbia precedenza su $\vee$ e che
  - $\vee$ sia associativo a sinistra.
- La nuova grammatica, che non fa uso di zucchero sintattico (perchè equivalente all'originale), è in grado di generare l'albero sintattico associato all'espressione $\neg(a \vee b)$?
- Definire la sintassi concreta per le espressioni booleane, aggiungendo la produzione che introduce lo zucchero sintattico (parentesi).
- Costruire l'albero di derivazione per $\neg(a \vee b)$.

Esercizio: if-then-else

- Si consideri la seguente sintassi astratta per i comandi:
  - $C \rightarrow \mathit{if}\ B\ \mathit{then}\ C \mid \mathit{if}\ B\ \mathit{then}\ C\ \mathit{else}\ C \mid \ldots$
- La grammatica è ambigua? Suggerimento: considerate la stringa $\mathit{if}\ b_1\ \mathit{then}\ \mathit{if}\ b_2\ \mathit{then}\ c_1\ \mathit{else}\ c_2$
  - Decidere a quale ramo *then* associare il ramo *else* ! (di solito si assume che l'else si riferisca al then più vicino)
- Trovare una sintassi concreta (quindi non ambigua e che può far uso di zucchero sintattico), supponendo che
  - i comandi siano divisi in comandi "bilanciati" e "sbilanciati" (ovvero in cui esiste un *else* che chiude il *then*, oppure no)
  - si possa usare come zucchero sintattico *begin* e *end* in modo che un comando sbilanciato racchiuso tra begin-end sia un comando bilanciato.
- Costruire gli alberi di derivazione per
  - $\mathit{if}\ b_1\ \mathit{then}\ \mathit{if}\ b_2\ \mathit{then}\ c_1\ \mathit{else}\ c_2$ e per
  - $\mathit{if}\ b_1\ \mathit{then}\ \mathit{begin}\ \mathit{if}\ b_2\ \mathit{then}\ c_1\ \mathit{end}\ \mathit{else}\ c_2$